

Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

Batch: A-4

Roll No.: 16010122151

Experiment / Assignment / Tutorial No: 9

Title: Study Report on Features of any two Simulation Software

Objective: The objectives of the lab experiment are to compare and analyze the features of two simulation software tools in discrete event simulation, focusing on their modeling capabilities, user interface, and statistical analysis tools. Students will apply theoretical knowledge to create models, evaluate software performance, and document their findings in a structured report.

Expected Outcome of Experiment:

CO4: Analyze the systems for input modeling and validation.

CO5: Estimate the different parameters of absolute and relative performance of different simulation systems.

Books/ Journals/ Websites referred:

Tasks to be Performed:

Write-up Contents

1. Name of Simulation Software 1:

- List of at least 5 features
- Description of each feature (at least 05 sentences each)
- Screenshots of the exploration of Simulation Software
- Comment on usefulness of these features
- Limitations of this Software

2. Name of Simulation Software 2:

- List of at least 5 features
- Description of each feature (at least 05 sentences each)
- Screenshots of exploration of Simulation Software
- Comment on usefulness of these features
- Limitations of this Software

Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

Implementation:

1. Simulation Software - NetLogo

List of at least 5 features

1. Agent-Based Modeling (ABM) Framework
2. Integrated Graphical User Interface (GUI) and 2D/3D Visualization
3. Simple, Logo-based Programming Language
4. Extensive "Models Library"
5. BehaviorSpace Experimentation Tool

Description of each feature

1. **Agent-Based Modeling (ABM) Framework:** NetLogo is fundamentally an agent-based modeling (ABM) tool. This approach simulates the parallel actions of numerous 'agents' (like people, animals, or molecules) to observe emergent, large-scale patterns. Agents follow simple, individual rules, leading to complex system-level behavior that wasn't explicitly programmed. This method is ideal for modeling complex adaptive systems, such as social networks, traffic jams, or ecosystems. The environment itself is composed of 'patches' and an 'observer', which agents can also interact with.
2. **Integrated Graphical User Interface (GUI):** NetLogo operates as a self-contained application with an integrated graphical user interface (GUI). The 'Interface' tab lets modelers build a custom dashboard with interactive elements. These elements include buttons to run functions, sliders to adjust parameters like population, and plots for real-time data visualization. This interactivity is excellent for demonstrating models and testing hypotheses on the fly. Users can instantly observe the visual impact of changing a variable in the simulation 'world'.
3. **Simple, Logo-based Programming Language:** NetLogo uses a simple, Logo-based programming language designed to be accessible for beginners and non-programmers. Its syntax is English-like (e.g., ask turtles [move-to...]) and focuses on agent-level commands, lowering the barrier to entry. This allows students and researchers to focus on their simulation's logic rather than on complex programming. The application includes a 'Code' tab with a dedicated editor. This editor aids development with features like syntax highlighting and a built-in error checker.
4. **Extensive "Models Library":** A key feature is NetLogo's extensive 'Models Library,' which is bundled with the software. This library contains hundreds of pre-built, well-documented models spanning fields like social science, biology, physics, and economics. For learners, it serves as an excellent resource for exploring and modifying existing simulations to understand ABM concepts. For researchers, these models provide validated starting points, saving significant

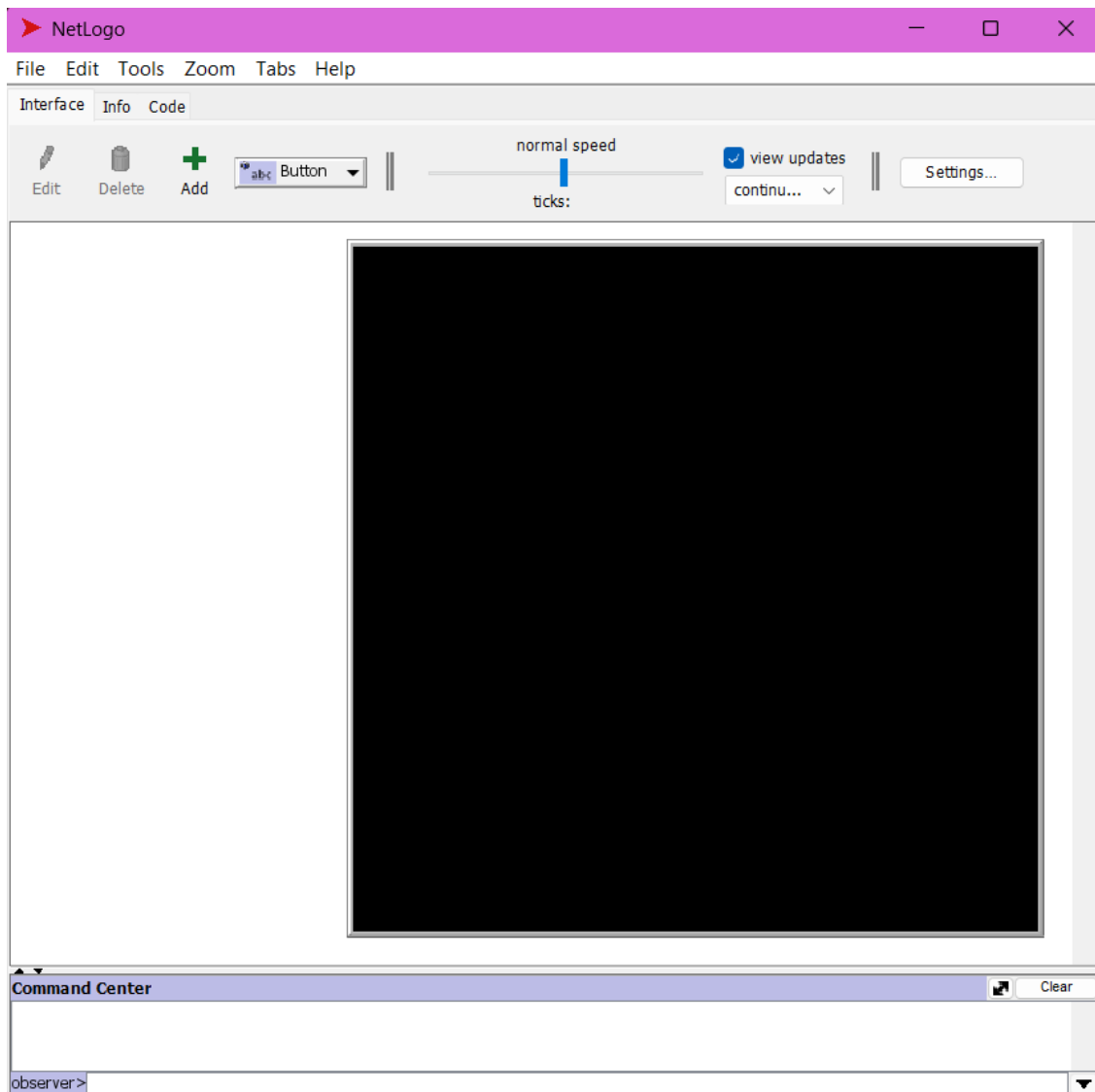
Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

development time. A user can, for example, instantly open and run the 'Wolf Sheep Predation' model to analyze a complex ecosystem.

5. **BehaviorSpace Experimentation Tool:** BehaviorSpace is NetLogo's integrated tool for rigorous scientific experimentation and statistical analysis. It enables users to run 'headless' simulations, meaning without the graphical interface, which allows experiments to run much faster. The tool automates the process of 'sweeping' through different parameter ranges, such as running a model 100 times while varying a population variable from 10 to 1,000. BehaviorSpace automatically executes all specified simulation runs. Finally, it collects the specified output data (like count turtles) and exports it into a structured format, such as a CSV or spreadsheet, for external statistical analysis.

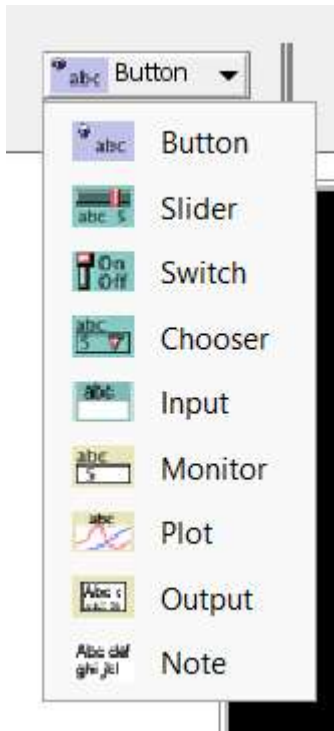
Screenshots of the exploration of Simulation Software

Home Interface:

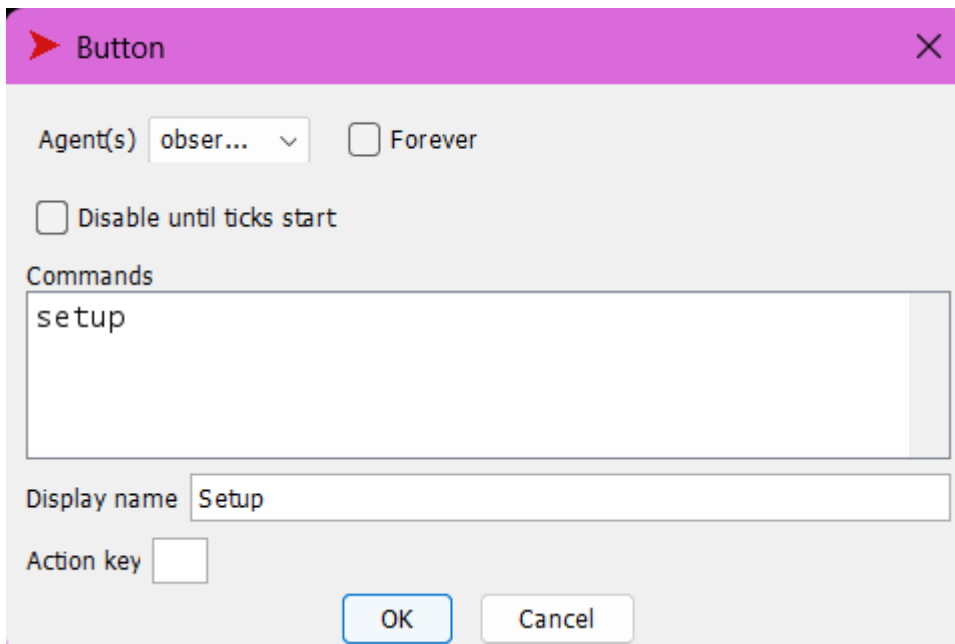


Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

Interactives:

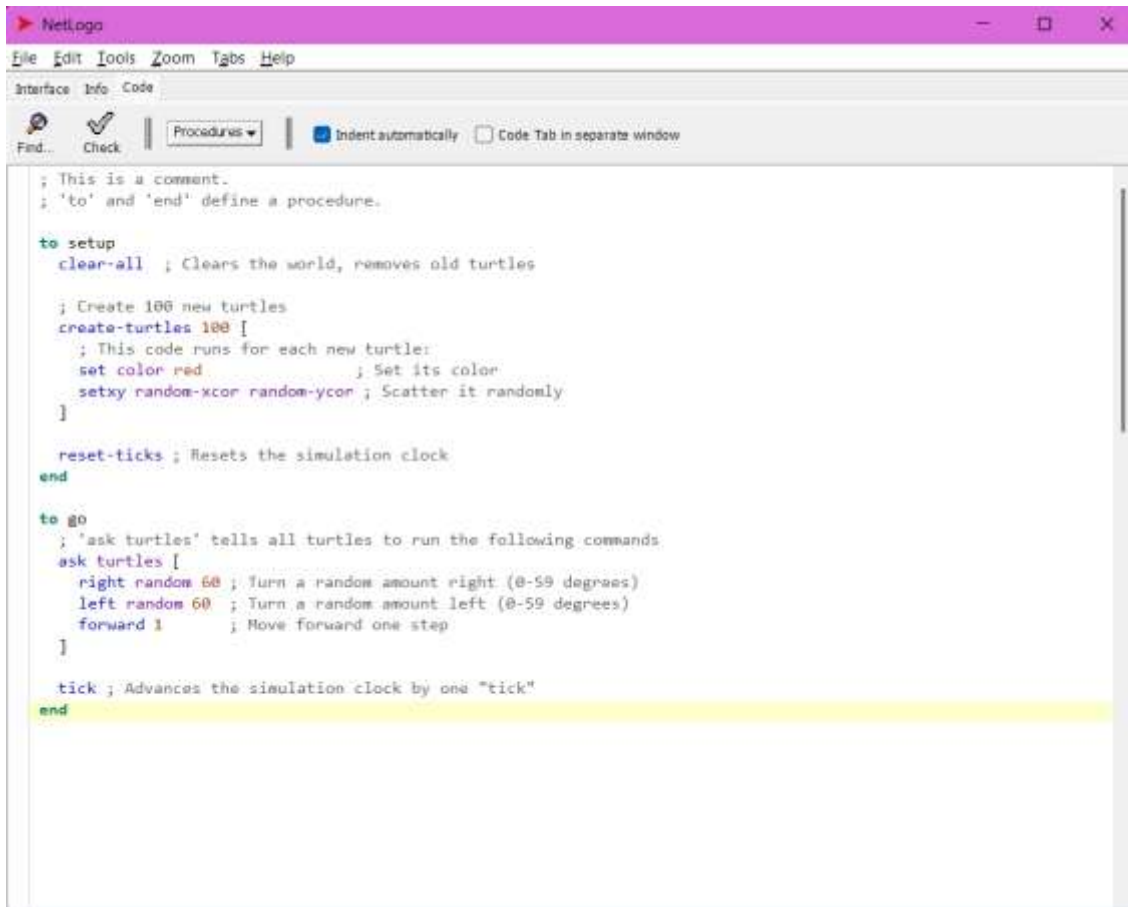


Setting Up button:



Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

Code Interface:

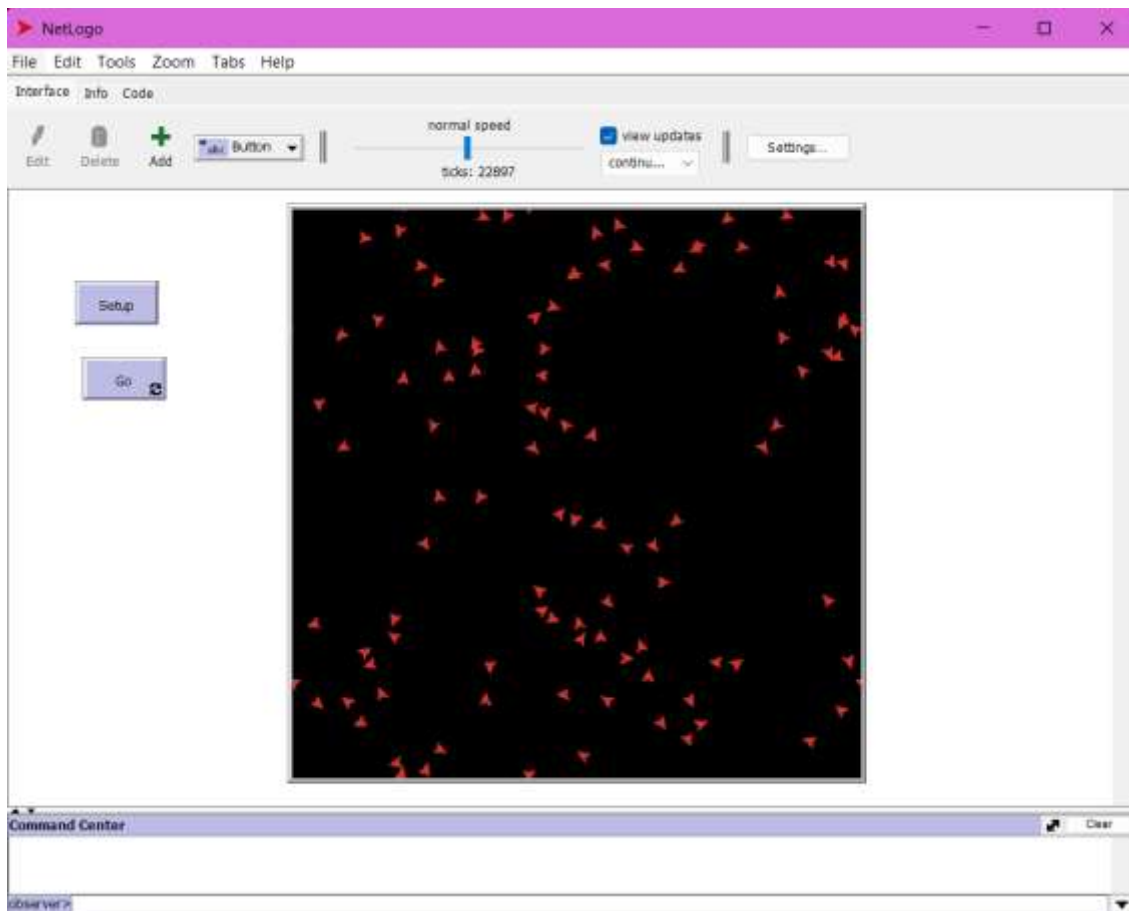


The screenshot shows the NetLogo code editor window. The title bar is purple and says "NetLogo". The menu bar includes File, Edit, Tools, Zoom, Tabs, and Help. Below the menu bar are tabs for Interface, Info, and Code. The Code tab is active, showing a script with the following code:

```
; This is a comment.  
; 'to' and 'end' define a procedure.  
  
to setup  
  clear-all ; Clears the world, removes old turtles  
  
  ; Create 100 new turtles  
  create-turtles 100 [  
    ; This code runs for each new turtle:  
    set color red ; Set its color  
    setxy random-xcor random-ycor ; Scatter it randomly  
  ]  
  
  reset-ticks ; Resets the simulation clock  
end  
  
to go  
  ; 'ask turtles' tells all turtles to run the following commands  
  ask turtles [  
    right random 60 ; Turn a random amount right (0-59 degrees)  
    left random 60 ; Turn a random amount left (0-59 degrees)  
    forward 1 ; Move forward one step  
  ]  
  
  tick ; Advances the simulation clock by one "tick"  
end
```

Simple Simulation Run:

Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)



Comment on usefulness of these features

The features of NetLogo combine to create an environment that is incredibly useful for both learning and research. The **GUI and simple language** make it accessible, allowing users to build and test models quickly. The visualization is not just for show; it's a powerful debugging and analysis tool, as you can *see* if your agents are behaving as expected. The **Models Library** is a huge asset for education and for jump-starting new projects. Finally, **BehaviorSpace** turns NetLogo from a simple "toy" into a serious scientific tool, enabling the rigorous, data-driven analysis.

Limitations of this Software

1. **Performance:** NetLogo is not built for high-performance computing. It can slow down significantly when handling hundreds of thousands or millions of agents, making very large-scale simulations impractical.
2. **Language Scope:** The simplicity of the NetLogo language is also a limitation. It is not a general-purpose language, so performing complex data manipulation, integrating with external databases, or using advanced statistical libraries is difficult without external scripts.

Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

3. **Primary Focus:** It is heavily specialized for Agent-Based Modeling. While you can technically build other types of simulations (like discrete-event), it's often clumsy and not what the tool is designed for.

2. Simulation Software - SimPy (Python)

List of at least 5 features

1. Process-Based Discrete-Event Simulation (DES)
2. Python Library Integration
3. Resource Modeling (Resources, Containers, Stores)
4. Process Interaction and Event-Based Control
5. No Built-in GUI or Visualization

Description of each feature

1. **Process-Based Discrete-Event Simulation (DES):** SimPy is a library for discrete-event simulation (DES), which models a system as a sequence of events occurring at specific points in time. This is a different paradigm from NetLogo's agent-based approach. SimPy uses a 'process-based' methodology, modeling active entities like customers or cars as 'processes'. These processes can be paused while waiting for an event, such as a server becoming free, and then resumed. This approach makes it highly intuitive for modeling systems involving queues, resources, and factories.
2. **Python Library Integration:** As a standard Python library, SimPy integrates seamlessly with the entire Python data science ecosystem. This integration is arguably its most powerful feature, enabling sophisticated analysis. Users can leverage libraries like Pandas for data manipulation and Matplotlib or Seaborn for creating complex plots. For advanced statistical analysis and input modeling, libraries like SciPy and NumPy are fully available. This also allows the simulation to be driven by external data from APIs, databases, or Excel files.
3. **Resource Modeling (Resources, Containers, Stores):** SimPy provides built-in components to easily model common simulation constraints. The Resource object is used to model entities with limited capacity, like servers, machines, or employees. Processes can 'request' a resource and will automatically be queued if it's unavailable, waiting until it's 'released'. Container objects are provided to model continuous inventories, such as the fuel level in a tank. Similarly, Store objects model discrete inventories, like items in a warehouse.
4. **Process Interaction and Event-Based Control:** SimPy's event control is built on Python's **generators**, which allows processes to pause and resume. A process runs until it 'yields' an event, effectively pausing its execution. Examples of events include `env.timeout(10)` (waiting for time to pass) or `resource.request()` (waiting for a resource). The central Environment (`env`) manages the simulation clock and is responsible for resuming processes when their specific event

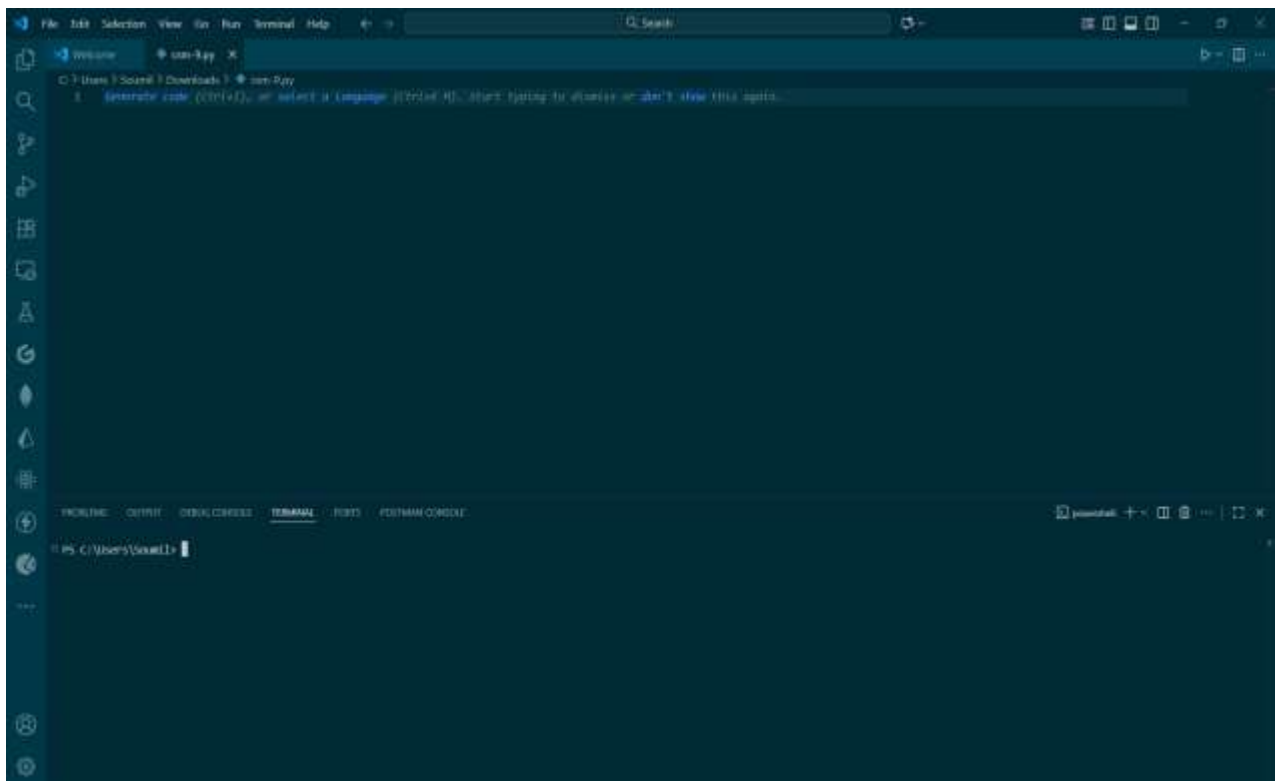
Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

occurs. This architecture also allows processes to trigger new events, enabling complex synchronization and interaction between them.

5. **No Built-in GUI or Visualization:** In a key contrast to NetLogo, SimPy provides no built-in graphical user interface (GUI) or visualization. It is a 'headless' library, designed to be run from a script or console. The primary output from a simulation is text printed to the console, such as event logs (e.g., "Car 1 arrived at time 5.0"). Alternatively, data is collected during the run and saved to a file for later analysis. All visualizations must be created after the simulation using external libraries, such as plotting queue length over time with Matplotlib.

Screenshots of the exploration of Simulation Software

VSCode Interface:



Using Terminal to install SimPy:

```
PS C:\Users\Soumil> pip install simpy
Defaulting to user installation because normal site-packages is not writeable
Collecting simpy
  Using cached simpy-4.1.1-py3-none-any.whl.metadata (6.1 kB)
Using cached simpy-4.1.1-py3-none-any.whl (27 kB)
Installing collected packages: simpy
Successfully installed simpy-4.1.1
```

Somaiya Vidyavihar University
(A Constituent College of Somaiya Vidyavihar University)

Simple Alice Bob Simulation:

```
--- Starting Simulation ---  
Alice speaking at time 0  
Bob speaking at time 0  
Alice speaking at time 2  
Bob speaking at time 3  
Alice speaking at time 4  
Bob speaking at time 6  
Alice speaking at time 6  
Alice speaking at time 8  
Bob speaking at time 9  
--- Simulation Finished ---
```

Comment on usefulness of these features

SimPy is a powerful and flexible tool, especially for users who are comfortable with Python. Its process-based design makes it feel natural when modeling things like queues or systems with limited resources. Built-in Resource objects are a huge time-saver—you don't have to write your own queue logic from scratch. What really sets SimPy apart is how seamlessly it works with Python, giving you full control over inputs, complex behaviors, and data analysis. Plus, since it doesn't rely on a graphical interface, simulations run fast and can easily be integrated into larger server-based applications.

Where SimPy Falls Short

1. **Tough for Beginners:** You need solid Python skills, especially around generators (yield), which can be confusing if you're just starting out.
2. **No Visuals:** There's no built-in way to see what's happening in your simulation. If something goes wrong—like a car stuck in traffic—you won't spot it visually like you could in NetLogo. You'll have to rely on logs and code.
3. **Manual Data Tracking:** SimPy doesn't automatically collect stats. You'll need to write your own code to track things like wait times or resource usage, which adds extra effort.

Conclusion:

NetLogo and SimPy are great simulation tools, each with its own strength.

- **NetLogo** is visual and agent-based—perfect for modeling complex systems like ecosystems or crowds, especially if you prefer a user-friendly, all-in-one interface.
- **SimPy** is a Python library built for data-heavy, discrete-event simulations like queues or factories. It's ideal when you need deep analysis and want to tap into Python's broader ecosystem.

Simple breakdown: NetLogo shines in visuals, SimPy rules in data.